LANGUAGE AGNOSTIC

CHEAT SHEETS

A Complete Reference for Coding Concepts & Algorithms

Pseudocode · Big O Notation · Recursion · Data Structures · Sorting · Searching · Dynamic Programming · Divide & Conquer · Bitwise Ops · Design Principles · Problem Solving

14 REFERENCE SHEETS

CONTENTS

Covers Algorithms · Data Structures · Design · Patterns (2025)

Begin → Sheet 01

Table of Contents

14 reference sheets · algorithms · data structures · design

Sheet 01

Pseudocode & Conventions

dialect · keywords · syntax rules · translations · reading pseudocode

WHAT IS PSEUDOCODE?

Pseudocode is a structured, language-agnostic way to describe algorithms and program logic using plain English combined with a small set of reserved words. It is not executable — it is a *design tool* for thinking through logic before committing to a real programming language. Good pseudocode is precise enough to translate directly into any language, yet readable by anyone regardless of their coding background.

This sheet defines the **pseudocode dialect** used across all sheets in this set. Every reserved word is `ALL CAPS`. Assignment uses `SET … TO`. Blocks close with explicit `END` keywords. Comments use `//`. Everything else is plain English that describes intent clearly.

CORE WRITING RULES

- `ALL CAPS` — reserved words only; never for emphasis
- `SET x TO 5` — the only assignment syntax; no `=`
- Indent one level per nested block (2 or 4 spaces consistently)
- Close every block: `END IF`, `END FOR`, `END WHILE`, `END FUNCTION`
- `// comment` — inline or standalone; ignored during "execution"
- Use plain English for conditions: `IF score > 90 THEN`
- One action per line — never chain multiple statements on one line
- Prefer clarity over brevity — pseudocode is documentation, not code golf

LINE-BY-LINE NOTES

- `SET … TO` — assigns a value; use for all variables and updates throughout
- `PRINT` — outputs a value; use `+` to concatenate strings inline
- `IF / ELSE IF / ELSE / END IF` — full conditional chain; `END IF` is mandatory
- `FOR each … IN … DO` — iterates over a collection; index-based loops use `FOR i FROM 1 TO n DO`
- `END FOR` — closes the for block; never omit even for single-line bodies
- `WHILE … DO / END WHILE` — loops while condition is true; update the variable inside the body
- `FUNCTION name(params)` — declares a reusable block; params are comma-separated plain names
- `RETURN` — exits the function and passes a value back to the caller
- `CALL name(args)` — invokes a function; assign with `SET result TO CALL …`
- Nesting — indent each level consistently; close all open blocks before the next sibling

DATA & EXPRESSION CONVENTIONS

NOTATION	MEANING / EXAMPLE
`[1, 2, 3]`	Array / list literal
`{key: value}`	Dictionary / hash map literal
`"text"`	String literal (double quotes)
`TRUE / FALSE`	Boolean literals (ALL CAPS)
`NULL`	Absence of value
`+, −, *, /`	Standard arithmetic operators
`MOD`	Modulo (remainder): `7 MOD 3` → 1
`DIV`	Integer division: `7 DIV 3` → 2
`=, ≠, <, >, ≤, ≥`	Comparison operators
`AND, OR, NOT`	Logical operators (ALL CAPS)
`array[i]`	Index access (0-based assumed)
`LENGTH OF array`	Number of elements in a collection
`APPEND x TO list`	Add element to end of a list

RESERVED KEYWORDS

KEYWORD(S)	PURPOSE
`SET … TO`	Variable assignment and update
`PRINT`	Output a value to the reader
`IF / THEN`	Begin a conditional branch
`ELSE IF / ELSE`	Additional or fallback branches
`END IF`	Close an IF block
`FOR … IN … DO`	Iterate over a collection
`FOR … FROM … TO … DO`	Numeric range loop
`END FOR`	Close a FOR block
`WHILE … DO`	Loop while a condition is true
`END WHILE`	Close a WHILE block
`FUNCTION … END FUNCTION`	Define a reusable named procedure
`RETURN`	Exit function, optionally pass a value
`CALL`	Invoke a named function
`// comment`	Inline or standalone annotation

DIALECT IN ACTION — ANNOTATED EXAMPLE

```
// ── Variable assignment & output ──────────
SET name TO "Alice"
SET age TO 30
PRINT "Hello, " + name          // Hello, Alice

// ── Conditionals ──────────────────────────
IF age >= 18 THEN
  PRINT "Adult"
ELSE IF age >= 13 THEN
  PRINT "Teenager"
ELSE
  PRINT "Child"
END IF

// ── For loop (collection iteration) ───────
SET scores TO [88, 92, 75, 96]
SET total TO 0
FOR each score IN scores DO
  SET total TO total + score
END FOR
PRINT total / 4                 // 87.75

// ── While loop ────────────────────────────
SET count TO 1
WHILE count <= 5 DO
  PRINT count
  SET count TO count + 1
END WHILE                       // prints 1 2 3 4 5

// ── Function definition & call ────────────
FUNCTION greet(person)
  SET message TO "Hello, " + person
  RETURN message
END FUNCTION

SET result TO CALL greet("Bob")
PRINT result                    // Hello, Bob

// ── Nested logic inside a function ────────
FUNCTION classify(n)
  IF n > 0 THEN
    RETURN "positive"
  ELSE IF n < 0 THEN
    RETURN "negative"
  ELSE
    RETURN "zero"
  END IF
END FUNCTION
```

PSEUDOCODE → PYTHON → JAVASCRIPT → JAVA — SYNTAX TRANSLATION

PSEUDOCODE	PYTHON	JAVASCRIPT	JAVA
`SET x TO 5`	`x = 5`	`let x = 5;`	`int x = 5;`
`PRINT x`	`print(x)`	`console.log(x);`	`System.out.println(x);`
`IF x > 0 THEN … END IF`	`if x > 0: …`	`if (x > 0) { … }`	`if (x > 0) { … }`
`ELSE IF x = 0 THEN`	`elif x == 0:`	`else if (x === 0) {`	`else if (x == 0) {`
`FOR each item IN list DO`	`for item in list:`	`for (const item of list) {`	`for (Type item : list) {`
`FOR i FROM 1 TO 10 DO`	`for i in range(1, 11):`	`for (let i = 1; i <= 10; i++) {`	`for (int i = 1; i <= 10; i++) {`
`WHILE count < 10 DO`	`while count < 10:`	`while (count < 10) {`	`while (count < 10) {`
`FUNCTION add(a, b)`	`def add(a, b):`	`function add(a, b) {`	`int add(int a, int b) {`
`RETURN result`	`return result`	`return result;`	`return result;`
`SET r TO CALL add(3, 4)`	`r = add(3, 4)`	`const r = add(3, 4);`	`int r = add(3, 4);`
`APPEND x TO list`	`list.append(x)`	`list.push(x);`	`list.add(x);`
`LENGTH OF list`	`len(list)`	`list.length`	`list.size()`
`x MOD 2`	`x % 2`	`x % 2`	`x % 2`
`AND / OR / NOT`	`and / or / not`	`&& / || / !`	`&& / || / !`
`TRUE / FALSE / NULL`	`True / False / None`	`true / false / null`	`true / false / null`

COMMON MISTAKES

- Using `=` for assignment instead of `SET … TO` — breaks the language-agnostic goal
- Omitting `END IF` / `END FOR` — ambiguous nesting; always close every block explicitly
- Mixing real language syntax (e.g. Python's `elif`) with pseudocode — defeats the purpose
- Writing overly detailed pseudocode (memory allocation, exact API calls) — describe intent, not implementation

TIPS

- Write pseudocode before writing real code — it forces you to solve the logic problem independently of syntax
- Use comments (`//`) to explain *why*, not *what* — the keywords already say what is happening
- If someone unfamiliar with programming can read it aloud and understand it, it's good pseudocode
- For complex algorithms, pseudocode is the deliverable — teams can review logic without knowing the target language

SUMMARY

- `ALL CAPS` keywords — reserved words; everything else is plain English
- `SET … TO` — universal assignment; `CALL` — function invocation
- Explicit closers — `END IF`, `END FOR`, `END WHILE`, `END FUNCTION`
- Translation table maps pseudocode to Python, JavaScript, and Java for every core construct

Sheet 02

Big O Notation

time complexity · space complexity · growth rates · best/worst/average · common classes

WHAT IS BIG O?

Big O notation describes how the runtime or memory usage of an algorithm *scales* as the input size **n** grows toward infinity. It expresses the *upper bound* of growth — the worst case — and ignores constant factors and lower-order terms. Two algorithms with the same Big O class may have very different real-world performance for small n, but Big O tells you which one wins as data grows large.

Big O measures **time complexity** (number of operations) and **space complexity** (memory allocated) independently. An algorithm can be O(n) time but O(1) space, or vice versa. Always consider both dimensions when evaluating an algorithm for production use.

KEY RULES

- **Drop constants** — O(2n) simplifies to O(n)
- **Drop lower terms** — O(n² + n) simplifies to O(n²)
- **Different inputs → different variables** — O(a + b), not O(2n)
- **Nested loops multiply** — two nested loops over n → O(n²)
- **Sequential steps add** — two passes over n → O(n + n) → O(n)
- **Best ≠ Big O** — Big O is worst case; Ω (Omega) is best case; Θ (Theta) is tight average
- **Recursive calls** — count the call tree depth and branching factor

COMPLEXITY CLASSES — ORDERED BEST TO WORST

NOTATION	NAME	EXAMPLE ALGORITHM / OPERATION	N=10	N=100	N=1 000
O(1)	Constant	Array index lookup, hash map get	1	1	1
O(log n)	Logarithmic	Binary search, balanced BST lookup	3	7	10
O(n)	Linear	Linear search, single array traversal	10	100	1 000
O(n log n)	Linearithmic	Merge sort, heap sort, quicksort (avg)	33	664	9 966
O(n²)	Quadratic	Bubble sort, insertion sort, nested loops	100	10 000	1 000 000
O(n³)	Cubic	Naive matrix multiplication, triple nested loops	1 000	1 000 000	10^9
O(2ⁿ)	Exponential	Recursive Fibonacci, all subsets	1 024	10^{30}	10^{301}
O(n!)	Factorial	All permutations, travelling salesman brute force	3 628 800	10^{157}	∞ (practical)

COMMON DATA STRUCTURE OPERATIONS

STRUCTURE	ACCESS	SEARCH	INSERT	DELETE
Array	O(1)	O(n)	O(n)	O(n)
Linked List	O(n)	O(n)	O(1)	O(1)
Hash Map	—	O(1)*	O(1)*	O(1)*
Binary Search Tree	O(log n)*	O(log n)*	O(log n)*	O(log n)*
Stack / Queue	O(n)	O(n)	O(1)	O(1)
Heap (min/max)	O(1) top	O(n)	O(log n)	O(log n)

* Average case. Worst case for hash map is O(n) (all collisions); BST worst case is O(n) (unbalanced).

HOW TO ANALYSE PSEUDOCODE

PATTERN	COMPLEXITY
Single statement	O(1)
Loop 1 → n	O(n)
Loop halves each iteration	O(log n)
Loop inside loop (both 1→n)	O(n²)
Two sequential loops (not nested)	O(n)
Recursive: T(n) = T(n/2) + O(1)	O(log n)
Recursive: T(n) = 2·T(n/2) + O(n)	O(n log n)
Recursive: T(n) = 2·T(n-1)	O(2ⁿ)
Allocate array of size n	O(n) space
Recursive depth d, no extra alloc	O(d) space

COMMON PROBLEM PATTERNS — EXPECTED COMPLEXITIES

PATTERN	TIME	SPACE	NOTES / EXAMPLE PROBLEM
Single pass	O(n)	O(1)	Find max / min, running sum, prefix sum build
Two pointers	O(n)	O(1)	Pair sum in sorted array, palindrome check, container with most water
Sliding window	O(n)	O(k)	Longest substring without repeat, min window substring
Binary search	O(log n)	O(1)	Search sorted array; also binary-search on the answer space (e.g. capacity problems)
Hash map / set	O(n)	O(n)	Two-sum, anagram grouping, first duplicate, frequency count
BFS	O(V + E)	O(V)	Shortest path (unweighted), level-order traversal, rotten oranges
DFS / backtracking	O(V + E) / O(b^d)	O(depth)	Connected components, all paths, subsets, permutations, N-queens
Heap / priority queue	O(n log k)	O(k)	Top-k elements, merge k sorted lists, median of stream
Sorting first	O(n log n)	O(1)–O(n)	Merge intervals, group anagrams, meeting rooms
Divide & conquer	O(n log n)	O(log n)	Merge sort, quickselect, closest pair of points
Dynamic programming (1-D)	O(n)	O(n)	Fibonacci, climbing stairs, house robber, longest increasing subsequence
Dynamic programming (2-D)	O(n · m)	O(n · m)	Longest common subsequence, edit distance, 0/1 knapsack, coin change

COMMON MISTAKES

- Forgetting to drop constants — O(3n) is just O(n)
- Assuming O(1) hash map access is always true — collisions make it O(n) worst case
- Confusing Big O (worst) with average case — quicksort is O(n log n) avg but O(n²) worst
- Ignoring space complexity — an O(n) time algorithm may still fail with O(n²) space

TIPS

- Count loops first — one loop = O(n), nested loops = multiply their bounds
- For recursion, draw the call tree — branches × depth gives you the class
- When two algorithms have the same Big O, measure empirically for small n — constants matter in practice
- O(n log n) is often the best achievable for comparison-based sorting — know why

SUMMARY

- Big O = upper bound on growth as n → ∞; drop constants and lower terms
- O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ) < O(n!)
- Nested loops multiply; sequential loops add (then simplify)
- Always consider both time *and* space complexity

Recursion

WHAT IS RECURSION?

A function is **recursive** when it calls itself as part of its own definition. Every recursive solution needs exactly two parts: a **base case** that stops the recursion and returns a concrete value, and a **recursive case** that reduces the problem toward the base case with each call.

Each call pushes a new *stack frame* onto the call stack. If no base case is ever reached, the stack grows until a **stack overflow** error terminates the program. Recursion trades explicit loop logic for elegant problem decomposition — it excels at problems with naturally recursive structure: trees, graphs, divide-and-conquer, and combinatorics.

ANATOMY OF A RECURSIVE FUNCTION

- **Base case** — the simplest input where the answer is known directly; must always be reachable
- **Recursive case** — calls itself with a *smaller* or *simpler* version of the input
- **Progress** — each call must move closer to the base case; otherwise → infinite recursion
- **Return value** — each frame returns to its caller; the final answer bubbles back up the stack
- **Call depth** — equals the number of stack frames; typically O(n) or O(log n)
- **Tail recursion** — recursive call is the very last operation; some languages optimise this to O(1) space

NOTES

- → `IF n <= 1 THEN RETURN 1` — base case halts recursion; without it the function never stops
- → `n * CALL factorial(n-1)` — recursive case; n shrinks by 1 each call → guaranteed to reach base
- → factorial(5) unwinds as: 5×4×3×2×1 after all frames return
- → fib(n) has two recursive calls — call tree doubles each level → O(2ⁿ) without memoization
- → Binary search halves the search space each call — O(log n) depth, O(log n) stack space
- → `acc` in sumArr carries the running total — tail recursion pattern; no work done on return
- → Tail recursive calls can be converted to loops by compilers/interpreters that support TCO

RECURSION PATTERNS

PATTERN	DESCRIPTION	EXAMPLE
Linear	One recursive call per frame	factorial, linked list traversal
Binary	Two recursive calls per frame	fibonacci, binary tree traversal
Tail	Recursive call is the last operation	sumArr with accumulator
Mutual	Function A calls B; B calls A	isEven/isOdd, recursive descent parsers
Divide & Conquer	Split problem, solve halves, merge	merge sort, quicksort
Backtracking	Recurse into choices, undo on failure	maze solver, N-queens
Tree / Graph	Visit node, recurse into children	DFS, pre/in/post-order traversal

RECURSION VS ITERATION

DIMENSION	RECURSION	ITERATION
Readability	High for tree/graph problems	High for linear problems
Space	O(depth) call stack	O(1) with no extra structure
Overflow risk	Yes — deep recursion → crash	No
TCO support	Language-dependent	Always O(1) space
State tracking	Implicit in call stack	Explicit in variables
Best for	Trees, D&C, backtracking	Arrays, counters, streams

CLASSIC EXAMPLES IN PSEUDOCODE

```
// ── Factorial: n! = n × (n-1)! ──────────────────
FUNCTION factorial(n)
  IF n <= 1 THEN          // base case
    RETURN 1
  END IF
  RETURN n * CALL factorial(n - 1)  // recursive case
END FUNCTION

PRINT CALL factorial(5)         // 120
// Call stack: fact(5)→fact(4)→fact(3)→fact(2)→fact(1)

// ── Fibonacci: fib(n) = fib(n-1) + fib(n-2) ─────
FUNCTION fib(n)
  IF n <= 1 THEN
    RETURN n          // base cases: fib(0)=0, fib(1)=
  END IF
  RETURN CALL fib(n - 1) + CALL fib(n - 2)
END FUNCTION          // O(2^n) — very slow; see memoiz

// ── Binary search (recursive) ────────────────────
FUNCTION bSearch(arr, target, lo, hi)
  IF lo > hi THEN
    RETURN -1         // base case: not found
  END IF
  SET mid TO (lo + hi) DIV 2
  IF arr[mid] = target THEN
    RETURN mid
  ELSE IF arr[mid] < target THEN
    RETURN CALL bSearch(arr, target, mid + 1, hi)
  ELSE
    RETURN CALL bSearch(arr, target, lo, mid - 1)
  END IF
END FUNCTION          // O(log n) time, O(log n) space

// ── Sum of array (tail-recursive style) ──────────
FUNCTION sumArr(arr, i, acc)
  IF i = LENGTH OF arr THEN
    RETURN acc        // base case: accumulator holds
  END IF
  RETURN CALL sumArr(arr, i + 1, acc + arr[i])
END FUNCTION
```

COMMON MISTAKES

- Missing or unreachable base case — stack grows forever until overflow
- Recursive case does not reduce the problem — passing the same n causes infinite recursion
- Assuming recursion is always slow — O(log n) recursive binary search beats O(n) linear search
- Forgetting stack space — deep recursion on large inputs can crash even with correct logic

TIPS

- Write the base case first — get the simplest input right before thinking about recursion
- Trust the recursive call — assume it works correctly on the smaller input and build on that
- Draw the call tree for binary recursion — it reveals the exponential blowup quickly
- If recursion is correct but slow, add memoization (Sheet 11) before converting to iteration

SUMMARY

- Every recursive function needs a **base case** and a **recursive case**
- Each call adds a stack frame — depth determines space complexity
- Binary recursion (two calls) → O(2ⁿ) without memoization
- Tail recursion eliminates stack growth when the language supports TCO

Sheet 04

Arrays vs Linked Lists

contiguous memory · nodes & pointers · singly / doubly linked · random access · insertion cost

TWO FUNDAMENTAL SEQUENCE STRUCTURES

An **array** stores elements in a contiguous block of memory. Because each element occupies a fixed offset from the start address, any element can be read in O(1) time with a simple index calculation. The trade-off: inserting or deleting in the middle requires shifting all subsequent elements — O(n).

A **linked list** stores elements in nodes scattered anywhere in memory. Each node holds a value and a pointer to the next node (and optionally the previous node in a *doubly linked* list). There is no O(1) random access — reaching element k requires following k pointers from the head — but insertion and deletion at a known position are O(1) because only pointers change, not the surrounding elements.

KEY PROPERTIES

- **Array** — fixed or dynamic size; O(1) read; O(n) insert/delete mid; cache-friendly
- **Singly linked** — node has `value` + `next`; O(n) access; O(1) insert at head
- **Doubly linked** — node has `value` + `next` + `prev`; O(1) delete with node reference
- **Dynamic array** — doubles capacity when full; amortised O(1) append
- **Cache locality** — arrays benefit from CPU prefetching; linked lists scatter in memory
- **Memory overhead** — linked list stores pointers per node; arrays store data only
- **Use arrays** when random access or cache performance matters
- **Use linked lists** when frequent insertion/deletion mid-list with a known pointer

NOTES

- → `arr[2]` — index = base address + (2 × element size); always O(1)
- → Insert at index 2 shifts elements 4→5, 3→4, 2→3 before writing — worst case O(n)
- → `{ value, next }` — a node is just a record; `next` is a reference to the next node or NULL
- → `prepend` — sets new node's next to old head, returns new head; no traversal needed → O(1)
- → `printList` — follows `next` pointers until NULL; visits every node once → O(n)
- → `deleteAfter` — rewires one pointer; the removed node is garbage collected → O(1)
- → `find` — must walk from head; no shortcut → O(n); this is why linked lists have no random access

COMPLEXITY COMPARISON

OPERATION	ARRAY	SINGLY LINKED	DOUBLY LINKED
Access by index	O(1)	O(n)	O(n)
Search by value	O(n)	O(n)	O(n)
Insert at head	O(n)	O(1)	O(1)
Insert at tail	O(1)*	O(n)**	O(1)**
Insert at index i	O(n)	O(n)	O(n)
Delete at head	O(n)	O(1)	O(1)
Delete with ref	O(n)	O(n)***	O(1)
Memory per element	1 word	2 words	3 words

* Amortised for dynamic arrays. ** With tail pointer. *** Needs prev node.

CHOOSING THE RIGHT STRUCTURE

SITUATION	CHOOSE
Frequent random access by index	Array
Cache performance / iteration speed	Array
Fixed or known maximum size	Array
Many insertions/deletions at front	Linked List (singly)
Need O(1) delete given a pointer	Doubly Linked List
Implementing a stack	Array or Singly Linked
Implementing a queue	Doubly Linked or Circular Array
Unknown / highly variable size	Dynamic Array (most languages)

PSEUDOCODE — STRUCTURES & OPERATIONS

```
// ── Array operations ──────────────────────────
SET arr TO [10, 20, 30, 40, 50]
PRINT arr[2]              // 30  — O(1) random access

// Insert 99 at index 2 (must shift elements right)
FOR i FROM LENGTH OF arr - 1 TO 2 DO
  SET arr[i + 1] TO arr[i]
END FOR
SET arr[2] TO 99          // O(n) — shifted 3 elements

// ── Singly Linked List node ───────────────────
// A node is a structure: { value, next }
// head → [10|→] → [20|→] → [30|null]

FUNCTION createNode(val)
  RETURN { value: val, next: NULL }
END FUNCTION

// Prepend to head — O(1)
FUNCTION prepend(head, val)
  SET newNode TO CALL createNode(val)
  SET newNode.next TO head
  RETURN newNode              // new head
END FUNCTION

// Traverse — O(n)
FUNCTION printList(head)
  SET current TO head
  WHILE current ≠ NULL DO
    PRINT current.value
    SET current TO current.next
  END WHILE
END FUNCTION

// Delete node after a given node — O(1)
FUNCTION deleteAfter(node)
  IF node.next ≠ NULL THEN
    SET node.next TO node.next.next
  END IF
END FUNCTION

// Find by value — O(n)
FUNCTION find(head, target)
  SET current TO head
  WHILE current ≠ NULL DO
    IF current.value = target THEN
      RETURN current
    END IF
    SET current TO current.next
  END WHILE
  RETURN NULL
END FUNCTION
```

COMMON MISTAKES

- Assuming linked list insert is always O(1) — it's O(n) if you must find the position first
- Off-by-one when shifting array elements — shift right-to-left on insert, left-to-right on delete
- Losing the head pointer when prepending — always save the new node first, then update head
- Using a linked list when random access is frequent — O(n) per access will dominate runtime

TIPS

- In most modern languages, the default "list" is a dynamic array — use it unless you specifically need linked-list properties
- Keep a `tail` pointer in linked lists when you need O(1) append
- Sentinel (dummy) head nodes simplify edge cases: empty list and single-element list behave identically
- Two-pointer technique (fast/slow) on linked lists solves cycle detection and middle-finding in O(n) O(1) space

SUMMARY

- Array = contiguous memory; O(1) access, O(n) insert/delete mid
- Linked list = nodes + pointers; O(n) access, O(1) insert/delete with reference
- Doubly linked adds a `prev` pointer — enables O(1) delete given the node
- Cache locality favours arrays for iteration; linked lists pay pointer-chasing overhead

Sheet 05

Stacks & Queues

LIFO · FIFO · push/pop · enqueue/dequeue · deque · monotonic stack · applications

CONSTRAINED ACCESS STRUCTURES

A **stack** enforces *Last In, First Out* (LIFO): the most recently added element is the first one removed. Think of a stack of plates — you can only take from the top. All operations (**push**, **pop**, **peek**) operate on one end and run in O(1).

A **queue** enforces *First In, First Out* (FIFO): elements are added at the back (**enqueue**) and removed from the front (**dequeue**). Think of a line at a counter — the first person to arrive is the first served. All operations are O(1) with the right implementation. Both structures are implemented with an array or linked list underneath; the abstraction is the constrained access policy.

CORE OPERATIONS

- **Stack push(x)** — add x to top; O(1)
- **Stack pop()** — remove and return top; O(1); error if empty
- **Stack peek()** — read top without removing; O(1)
- **Queue enqueue(x)** — add x to back; O(1)
- **Queue dequeue()** — remove and return front; O(1)
- **Queue front()** — read front without removing; O(1)
- **isEmpty()** — returns TRUE if no elements; O(1); always check before pop/dequeue
- **Deque** — double-ended queue; push/pop at both ends in O(1)

NOTES

- → `push` / `pop` — both operate on the same end (top); LIFO order guaranteed
- → `peek` — reads without removing; safe to call anytime; does not change size
- → `enqueue` adds to back; `dequeue` removes from front — opposite ends enforce FIFO
- → isBalanced uses a stack: open brackets push, close brackets pop and check for a matching opener
- → Stack is empty at the end only if every opener was matched — empty stack = balanced
- → BFS uses a queue to process nodes level by level — nearest neighbours always dequeued first
- → `visited` set prevents revisiting — without it BFS loops on cyclic graphs

CLASSIC APPLICATIONS

STRUCTURE	APPLICATION
Stack	Function call stack / recursion management
Stack	Undo / redo in editors
Stack	Balanced bracket / parenthesis checking
Stack	Infix → postfix expression conversion
Stack	DFS (depth-first search) iterative version
Stack	Monotonic stack — next greater element
Queue	BFS (breadth-first search)
Queue	Task scheduling / CPU job queues
Queue	Print spooler, request buffers
Deque	Sliding window maximum
Priority Queue	Dijkstra's algorithm, A*, event simulation

IMPLEMENTATION TRADE-OFFS

BACKED BY	STACK FIT	QUEUE FIT	NOTES
Dynamic array	✔	Partial	Dequeue from front is O(n); use circular buffer
Circular array	✔	✔	O(1) both ends; fixed capacity or resize needed
Singly linked list	✔	✔*	O(1) push/pop head; need tail pointer for queue
Doubly linked list	✔	✔	O(1) both ends; more memory per node

MONOTONIC STACK PATTERN

GOAL	STACK ORDER
Next greater element	Decreasing (pop when current > top)
Next smaller element	Increasing (pop when current < top)
Largest rectangle in histogram	Increasing heights

PSEUDOCODE — IMPLEMENTATIONS & APPLICATIONS

```
// ── Stack (array-backed) ──────────────────────────
SET stack TO []
CALL stack.push(1)            // [1]
CALL stack.push(2)            // [1, 2]
CALL stack.push(3)            // [1, 2, 3]
PRINT CALL stack.peek()       // 3 — top, not removed
PRINT CALL stack.pop()        // 3 — removed; stack=[1,2]

// ── Queue (array-backed, conceptual) ──────────────
SET queue TO []
CALL queue.enqueue("a")       // ["a"]
CALL queue.enqueue("b")       // ["a","b"]
CALL queue.enqueue("c")       // ["a","b","c"]
PRINT CALL queue.dequeue()    // "a" — front removed
PRINT CALL queue.front()      // "b" — peek front

// ── Application: balanced parentheses ─────────────
FUNCTION isBalanced(str)
  SET stack TO []
  FOR each ch IN str DO
    IF ch = "(" OR ch = "[" OR ch = "{" THEN
      CALL stack.push(ch)
    ELSE IF ch = ")" OR ch = "]" OR ch = "}" THEN
      IF CALL stack.isEmpty() THEN
        RETURN FALSE
      END IF
      SET top TO CALL stack.pop()
      IF NOT CALL matches(top, ch) THEN
        RETURN FALSE
      END IF
    END IF
  END FOR
  RETURN CALL stack.isEmpty()
END FUNCTION

// ── Application: BFS with queue ───────────────────
FUNCTION bfs(graph, start)
  SET visited TO {}
  SET queue TO []
  CALL queue.enqueue(start)
  CALL visited.add(start)
  WHILE NOT CALL queue.isEmpty() DO
    SET node TO CALL queue.dequeue()
    PRINT node
    FOR each neighbour IN graph[node] DO
      IF NOT CALL visited.contains(neighbour) THEN
        CALL visited.add(neighbour)
        CALL queue.enqueue(neighbour)
      END IF
    END FOR
  END WHILE
END FUNCTION
```

COMMON MISTAKES

- Popping from an empty stack — always check `isEmpty()` first
- Using an array as a queue with `dequeue` at index 0 — that is O(n) shift; use a circular buffer or deque
- Forgetting to mark nodes visited in BFS — causes infinite loops on graphs with cycles
- Confusing DFS (stack) with BFS (queue) — they produce different traversal orders

TIPS

- Whenever you see "most recent" or "undo", think stack; whenever you see "order preserved" or "level by level", think queue
- Implement DFS iteratively with an explicit stack to avoid recursion depth limits
- A monotonic stack solves next-greater / next-smaller problems in O(n) — each element is pushed and popped at most once
- Two stacks can simulate a queue — amortised O(1) per operation

SUMMARY

- Stack = LIFO; push/pop/peek at top — O(1) all operations
- Queue = FIFO; enqueue at back, dequeue at front — O(1) all operations
- BFS uses queue; DFS uses stack (or call stack implicitly)
- Deque generalises both; monotonic stack is a constrained variant for range queries

Sheet 06

Hash Maps & Dictionaries

hashing · key-value pairs · collision handling · load factor · sets · frequency counting

HOW HASH MAPS WORK

A **hash map** (also called dictionary, associative array, or object) stores *key* → *value* pairs and provides average O(1) lookup, insertion, and deletion. Internally it maintains an array of *buckets*. A **hash function** maps any key to an integer index into that array. Ideally, different keys map to different buckets — but when two keys map to the same index, a **collision** occurs.

Collisions are resolved by *chaining* (each bucket holds a linked list of entries) or *open addressing* (probe to the next empty bucket). A high **load factor** (number of entries ÷ number of buckets) increases collision probability, so good implementations *rehash* — doubling the bucket array when the load factor exceeds a threshold (typically 0.75).

KEY CONCEPTS

- **Hash function** — deterministic; same key always → same index; fast to compute
- **Collision** — two keys produce the same bucket index; unavoidable in theory
- **Chaining** — bucket holds a list; worst case O(n) if all keys collide
- **Open addressing** — probe sequence finds next open slot; compact but deletion is tricky
- **Load factor** — entries / buckets; keep below 0.75 for good average-case performance
- **Rehashing** — rebuild the table at 2× size; amortised O(1) cost per insert
- **Key requirements** — keys must be hashable and comparable; mutable keys break lookups
- **Sets** — a hash map where only keys are stored (no values); same O(1) operations

NOTES

- → `map["key"] TO value` — inserts if absent, updates if present; both O(1) average
- → `"key" IN map` — O(1) membership test; safer than accessing a missing key (which errors or returns NULL)
- → `getOrDefault` — guards against missing key; pattern used constantly in freq counting
- → `countFreq` — the most common hash map pattern; transforms a list into a frequency table in O(n)
- → `twoSum` — stores each number's index as it scans; O(n) single pass vs O(n²) brute force
- → `complement IN seen` — if the other half of the pair was already seen, we have our answer
- → `groupAnagrams` — sorted characters form a canonical key; all anagrams share the same key

OPERATION COMPLEXITY

OPERATION	AVERAGE	WORST CASE
`get(key)`	O(1)	O(n)
`put(key, val)`	O(1)	O(n)
`delete(key)`	O(1)	O(n)
`contains(key)`	O(1)	O(n)
`iterate all pairs`	O(n)	O(n)
`rehash`	O(n)	O(n)

Worst case occurs when all keys hash to the same bucket (adversarial input or poor hash function).

COMMON HASH MAP PATTERNS

PATTERN	USE CASE
`Frequency count`	Count occurrences of items in an array
`Two-sum / complement`	Find pairs that satisfy a condition in O(n)
`Grouping by key`	Group anagrams, group by category
`Memoization cache`	Store computed results to avoid recomputation
`Existence check`	Replace O(n) linear search with O(1) lookup
`Graph adjacency`	Map node → list of neighbours
`Sliding window`	Track char/element counts within a window
`Deduplication`	Use a set to collect unique items in O(n)

PSEUDOCODE — OPERATIONS & PATTERNS

```
// ── Basic operations ──────────────────────────────
SET map TO {}
SET map["alice"] TO 42       // insert / update
SET map["bob"] TO 17
PRINT map["alice"]           // 42 — O(1) average
PRINT "alice" IN map         // TRUE
CALL map.delete("bob")       // remove key

// ── Safe get with default ─────────────────────────
FUNCTION getOrDefault(map, key, default)
  IF key IN map THEN
    RETURN map[key]
  END IF
  RETURN default
END FUNCTION

// ── Frequency count (classic pattern) ─────────────
FUNCTION countFreq(items)
  SET freq TO {}
  FOR each item IN items DO
    IF item IN freq THEN
      SET freq[item] TO freq[item] + 1
    ELSE
      SET freq[item] TO 1
    END IF
  END FOR
  RETURN freq
END FUNCTION
//countFreq(["a","b","a","c","b","a"]) → {a:3, b:2, c:1}

// ── Two-sum with hash map ─────────────────────────
FUNCTION twoSum(nums, target)
  SET seen TO {}
  FOR i FROM 0 TO LENGTH OF nums - 1 DO
    SET complement TO target - nums[i]
    IF complement IN seen THEN
      RETURN [seen[complement], i]
    END IF
    SET seen[nums[i]] TO i
  END FOR
  RETURN NULL
END FUNCTION
// twoSum([2,7,11,15], 9) → [0,1]

// ── Group anagrams ────────────────────────────────
FUNCTION groupAnagrams(words)
  SET groups TO {}
  FOR each word IN words DO
    SET key TO CALL sort(word)  // sorted chars as key
    IF key NOT IN groups THEN
      SET groups[key] TO []
    END IF
    CALL groups[key].append(word)
  END FOR
  RETURN CALL groups.values()
END FUNCTION
```

COMMON MISTAKES

- Using a mutable object (list, array) as a key — mutating it after insertion makes the entry unreachable
- Not checking key existence before access — missing key may return NULL or throw an error depending on language
- Assuming order is preserved — standard hash maps are unordered; use an ordered map if insertion order matters
- Ignoring worst-case O(n) — adversarial inputs can degrade performance; use a good hash or a tree map for guarantees

TIPS

- Frequency counting (map → int) solves dozens of interview problems — master this pattern first
- When converting O(n²) brute force to O(n), ask "what would I need to look up?" — that becomes the hash map key
- Sets are hash maps without values — use them for O(1) membership tests and deduplication
- Sorted keys or canonical forms (sorted string) make powerful composite keys for grouping problems

SUMMARY

- Hash map = key → value; O(1) average get/put/delete via hash function + bucket array
- Collisions resolved by chaining or open addressing; load factor drives rehashing
- Sets are hash maps without values — same O(1) membership; use for deduplication
- Frequency count and two-sum are the two most common hash map algorithmic patterns

Trees & Graphs

nodes · edges · BFS · DFS · BST · directed/undirected · adjacency list · traversal orders

TREES AND GRAPHS

A **graph** is a set of *nodes* (vertices) connected by *edges*. Edges can be *directed* (one-way) or *undirected* (two-way), and may carry a *weight*. Graphs can have cycles. They model networks, maps, dependencies, and social connections.

A **tree** is a special acyclic, connected, undirected graph where exactly one path exists between any two nodes. Rooted trees add a designated *root* node; each non-root node has exactly one *parent* and zero or more *children*. A **binary tree** limits children to at most two (left and right). A **binary search tree (BST)** adds the ordering invariant: all left-subtree values < node < all right-subtree values, giving O(log n) search on balanced trees.

VOCABULARY

- **Root** — topmost node of a rooted tree; has no parent
- **Leaf** — node with no children
- **Height** — longest path from root to a leaf (edges)
- **Depth** — distance from root to a given node
- **Degree** — number of edges connected to a node
- **Adjacency list** — map of node → [neighbours]; space-efficient for sparse graphs
- **Adjacency matrix** — n×n boolean grid; O(1) edge check, O(n²) space
- **DAG** — directed acyclic graph; models task dependencies, topological sort
- **Cycle** — path that starts and ends at the same node

NOTES

- → `IF node = NULL THEN RETURN` — base case for every tree recursion; handles empty subtrees and leaves
- → Pre-order — visit root first; used for copying a tree or serialising structure
- → In-order on a BST — visits nodes in sorted ascending order; use for sorted output
- → Post-order — visit children before parent; used for deletion (can't delete parent before children)
- → Level-order / BFS — uses a queue; processes all nodes at depth d before depth d+1
- → `visited` set in graph DFS — prevents revisiting nodes in cyclic graphs; required for correctness
- → BST insert — recurse left if val is smaller, right if larger; returns root to allow tree construction

TREE TRAVERSAL SUMMARY

ORDER	SEQUENCE	KEY USE
Pre-order	Root → Left → Right	Copy tree, serialise, prefix expression
In-order	Left → Root → Right	BST sorted output, validate BST
Post-order	Left → Right → Root	Delete tree, evaluate expression tree
Level-order	BFS layer by layer	Shortest path in unweighted tree, level printing

BST OPERATIONS

OPERATION	BALANCED	UNBALANCED (WORST)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

GRAPH PROPERTIES & ALGORITHMS

PROPERTY / ALGORITHM	DETAILS
Undirected	Edges have no direction; A↔B means A→B and B→A
Directed (Digraph)	Edges have direction; A→B does not imply B→A
Weighted	Edges carry a cost; models distances, latency
BFS	Shortest path (unweighted); O(V+E)
DFS	Cycle detection, connected components; O(V+E)
Topological Sort	Linear order of DAG nodes; DFS post-order or Kahn's
Dijkstra's	Shortest path in weighted graph; O((V+E) log V)
Union-Find	Detect cycles, connected components; near O(1) ops

PSEUDOCODE — TREE & GRAPH TRAVERSALS

```
// ── Binary tree node: { value, left, right } ──────────────

// Pre-order DFS: root → left → right
FUNCTION preOrder(node)
  IF node = NULL THEN RETURN END IF
  PRINT node.value
  CALL preOrder(node.left)
  CALL preOrder(node.right)
END FUNCTION

// In-order DFS: left → root → right (BST → sorted output)
FUNCTION inOrder(node)
  IF node = NULL THEN RETURN END IF
  CALL inOrder(node.left)
  PRINT node.value
  CALL inOrder(node.right)
END FUNCTION

// Post-order DFS: left → right → root (delete tree, eval expr)
FUNCTION postOrder(node)
  IF node = NULL THEN RETURN END IF
  CALL postOrder(node.left)
  CALL postOrder(node.right)
  PRINT node.value
END FUNCTION

// BFS (level-order) using a queue
FUNCTION levelOrder(root)
  IF root = NULL THEN RETURN END IF
  SET queue TO [root]
  WHILE NOT CALL queue.isEmpty() DO
    SET node TO CALL queue.dequeue()
    PRINT node.value
    IF node.left ≠ NULL THEN CALL queue.enqueue(node.left) END IF
    IF node.right ≠ NULL THEN CALL queue.enqueue(node.right) END IF
  END WHILE
END FUNCTION

// Graph DFS (adjacency list, tracks visited)
FUNCTION dfs(graph, node, visited)
  CALL visited.add(node)
  PRINT node
  FOR each neighbour IN graph[node] DO
    IF NOT CALL visited.contains(neighbour) THEN
      CALL dfs(graph, neighbour, visited)
    END IF
  END FOR
END FUNCTION

// BST insert
FUNCTION bstInsert(root, val)
  IF root = NULL THEN RETURN CALL createNode(val) END IF
  IF val < root.value THEN
    SET root.left TO CALL bstInsert(root.left, val)
  ELSE
    SET root.right TO CALL bstInsert(root.right, val)
  END IF
  RETURN root
END FUNCTION
```

COMMON MISTAKES

- Forgetting NULL/base case in tree recursion — causes null pointer errors on empty subtrees
- Not using a visited set in graph traversal — infinite loops on cyclic graphs
- Confusing height and depth — height measures from node down to leaf; depth measures from root down to node
- Assuming a BST is balanced — an insertion-order sorted sequence creates an O(n) chain (degenerate tree)

TIPS

- Most tree problems are solved by DFS recursion — write the base case first, then trust the recursive call
- BFS gives shortest path in unweighted graphs — always think BFS when you need "minimum steps"
- In-order traversal of a BST gives sorted output — use this to validate BST properties
- For balanced BSTs in production, prefer AVL or Red-Black trees (or use the language's built-in sorted map)

SUMMARY

- Tree = acyclic connected graph with a root; graph = general nodes + edges (may have cycles)
- DFS uses a stack (recursive or explicit); BFS uses a queue
- BST: left < node < right; O(log n) balanced, O(n) worst
- Always track visited nodes in graph traversal to handle cycles

Sorting Algorithms

bubble · selection · insertion · merge · quick · heap · counting · stability · in-place

SORTING FUNDAMENTALS

Sorting arranges elements in a defined order (typically ascending). The choice of algorithm depends on input size, whether the data is nearly sorted, memory constraints, and whether **stability** matters — a stable sort preserves the relative order of equal elements, which is essential when sorting by multiple keys.

There is a theoretical lower bound of **Ω(n log n)** for any *comparison-based* sorting algorithm — you cannot do better in the general case. Algorithms like counting sort and radix sort sidestep this by not comparing elements directly, achieving O(n + k) at the cost of a restricted key range. In practice, most languages use a hybrid: Timsort (Python, Java) or introsort (C++) — both O(n log n) average with tuned constants.

KEY PROPERTIES

- **Stable** — equal elements retain original relative order (merge sort, insertion sort, counting sort)
- **In-place** — sorts using O(1) extra space (bubble, selection, insertion, heap, quicksort)
- **Comparison-based** — only uses <, =, > comparisons; lower bound O(n log n)
- **Non-comparison** — counting sort, radix sort, bucket sort; can be O(n + k)
- **Adaptive** — exploits existing order; insertion sort is O(n) on nearly sorted data
- **Divide & Conquer** — merge sort and quicksort split the array recursively
- **n log n lower bound** — proven via decision tree argument; optimal for general comparison sort

NOTES

- Merge sort — divide array in half recursively until size 1, then merge sorted halves back up
- `merge` — two-pointer walk; always picks the smaller front element; O(n) per merge call
- Total work: O(log n) levels × O(n) merge per level = O(n log n); stable because `<=` keeps equal elements in order
- Quick sort — choose pivot (last element here), partition array so left < pivot < right, recurse
- `partition` — Lomuto scheme; i tracks the boundary of elements ≤ pivot; runs in O(n)
- Worst case O(n²) when pivot is always min/max — use random pivot or median-of-three to avoid
- Quick sort is in-place (O(log n) stack space); merge sort needs O(n) auxiliary array

ALGORITHM COMPARISON

ALGORITHM	BEST	AVERAGE	WORST	SPACE	STABLE
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	✔
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	✖
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	✔
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	✔
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	✖
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	✖
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	✔
Timsort	O(n)	O(n log n)	O(n log n)	O(n)	✔

SIMPLE SORTS — PSEUDOCODE SKETCHES

SORT	CORE IDEA
Bubble	Repeatedly swap adjacent out-of-order pairs; largest bubbles to end each pass
Selection	Find minimum of unsorted region, swap to front; n passes, n swaps total
Insertion	Grow a sorted prefix; insert each new element into its correct position by shifting
Counting	Count occurrences of each value k; reconstruct by iterating counts; requires bounded integer keys
Radix	Sort by each digit (LSD→MSD) using a stable sub-sort; O(d·n) where d = digit count
Heap	Build max-heap in O(n); repeatedly extract max to end; O(n log n) guaranteed, no extra space

MERGE SORT & QUICK SORT IN PSEUDOCODE

```
// ── Merge Sort — O(n log n) stable, O(n) extra space ──────────
FUNCTION mergeSort(arr)
  IF LENGTH OF arr <= 1 THEN
    RETURN arr
  END IF
  SET mid TO LENGTH OF arr DIV 2
  SET left TO CALL mergeSort(arr[0 .. mid - 1])
  SET right TO CALL mergeSort(arr[mid .. end])
  RETURN CALL merge(left, right)
END FUNCTION

FUNCTION merge(left, right)
  SET result TO []
  SET i TO 0
  SET j TO 0
  WHILE i < LENGTH OF left AND j < LENGTH OF right DO
    IF left[i] <= right[j] THEN
      CALL result.append(left[i])
      SET i TO i + 1
    ELSE
      CALL result.append(right[j])
      SET j TO j + 1
    END IF
  END WHILE
  // Append remaining elements
  WHILE i < LENGTH OF left DO
    CALL result.append(left[i])
    SET i TO i + 1
  END WHILE
  WHILE j < LENGTH OF right DO
    CALL result.append(right[j])
    SET j TO j + 1
  END WHILE
  RETURN result
END FUNCTION

// ── Quick Sort — O(n log n) avg, O(n²) worst, in-place ────────
FUNCTION quickSort(arr, lo, hi)
  IF lo >= hi THEN RETURN END IF
  SET p TO CALL partition(arr, lo, hi)
  CALL quickSort(arr, lo, p - 1)
  CALL quickSort(arr, p + 1, hi)
END FUNCTION

FUNCTION partition(arr, lo, hi)
  SET pivot TO arr[hi]
  SET i TO lo - 1
  FOR j FROM lo TO hi - 1 DO
    IF arr[j] <= pivot THEN
      SET i TO i + 1
      CALL swap(arr, i, j)
    END IF
  END FOR
  CALL swap(arr, i + 1, hi)
  RETURN i + 1
END FUNCTION
```

COMMON MISTAKES

- Using quicksort without random pivot on nearly sorted data — degrades to O(n²)
- Using a non-stable sort when secondary key order matters — swap to merge sort or Timsort
- Counting sort with large k (e.g. sorting strings by unicode) — O(k) space makes it impractical
- Off-by-one in merge's boundary indices — always verify lo, mid, hi ranges on paper first

TIPS

- Use the language's built-in sort in production — it's usually Timsort or introsort, highly optimised
- Insertion sort outperforms merge/quick sort for n < ~20 — this is why hybrid sorts use it for small partitions
- Stable + O(n log n) + simple to implement → merge sort; fastest in practice → quicksort with random pivot
- When keys are small integers (grades, ages, counts), counting sort runs in O(n) — use it

SUMMARY

- Comparison-sort lower bound is Ω(n log n) — no comparison algorithm can beat it
- Merge sort — stable, O(n log n) guaranteed, O(n) space; quicksort — in-place, O(n log n) avg, O(n²) worst
- Counting/radix sort — O(n) when keys are bounded integers; not comparison-based
- Stability preserves equal-element order — essential for multi-key sorts

Searching Algorithms

SEARCHING FUNDAMENTALS

Searching is the process of locating an item — or determining it is absent — within a data structure. The right algorithm depends on whether the data is *sorted*, how it is structured, and what you are searching for.

Linear search works on any unsorted collection in O(n). Once data is sorted, **binary search** reduces this to O(log n) by halving the search space each step. For graphs and trees, **BFS** (breadth-first) finds the shortest path in unweighted graphs while **DFS** (depth-first) explores deep paths and is used for cycle detection and connectivity. Beyond these, the **two-pointer** and **sliding window** techniques efficiently search for subarrays or pairs satisfying constraints.

CHOOSING A SEARCH STRATEGY

- **Unsorted array** → linear search O(n); no preprocessing required
- **Sorted array** → binary search O(log n); requires sorted data
- **Hash map / set** → O(1) lookup; best when data fits in memory
- **BST** → O(log n) balanced; supports range queries
- **Unweighted graph** → BFS for shortest path, DFS for connectivity
- **Weighted graph** → Dijkstra's (non-negative), Bellman-Ford (negative edges)
- **Sorted pairs / subarrays** → two-pointer pattern O(n)
- **Contiguous subarray** → sliding window pattern O(n)

NOTES

- Linear search — check every element; works on unsorted data; O(n) time, O(1) space
- Binary search — `mid = (lo + hi) DIV 2` avoids integer overflow vs `(lo+hi)/2` in fixed-width integers
- `lo <= hi` — the loop runs while the search space is non-empty; terminates in O(log n) iterations
- Two-pointer — works because array is sorted; moving left increases sum, moving right decreases it
- Two-pointer explores all candidate pairs in one O(n) pass without the O(n²) nested loop
- Sliding window — adds the new right element and drops the leftmost; window sum maintained in O(1) per step
- Sliding window avoids recomputing the sum from scratch each time — transforms O(n·k) brute force to O(n)

SEARCH ALGORITHM SUMMARY

ALGORITHM	TIME	SPACE	PRECONDITION
Linear Search	O(n)	O(1)	None
Binary Search	O(log n)	O(1)	Sorted array
Hash Map Lookup	O(1) avg	O(n)	Pre-built map
BST Search	O(log n)*	O(1)	Balanced BST
BFS	O(V+E)	O(V)	Graph + queue
DFS	O(V+E)	O(V)	Graph + stack
Two Pointer	O(n)	O(1)	Sorted or structured
Sliding Window	O(n)	O(1)–O(k)	Contiguous subarray

BINARY SEARCH VARIANTS

GOAL	CONDITION TO MOVE LO/HI
Find exact target	Return mid on match; lo=mid+1 or hi=mid-1 otherwise
First position ≥ target	If arr[mid] < target → lo=mid+1; else hi=mid; return lo
Last position ≤ target	If arr[mid] > target → hi=mid-1; else lo=mid; return hi
Search in rotated array	Determine which half is sorted; binary search that half
Find peak element	If arr[mid] < arr[mid+1] → lo=mid+1; else hi=mid
Search on answer space	Binary search over possible answers; validate with predicate

CORE SEARCH PATTERNS IN PSEUDOCODE

```
// ── Linear search — O(n) ──────────────────────────────
FUNCTION linearSearch(arr, target)
  FOR i FROM 0 TO LENGTH OF arr - 1 DO
    IF arr[i] = target THEN RETURN i END IF
  END FOR
  RETURN -1
END FUNCTION

// ── Binary search — O(log n); array must be sorted ────
FUNCTION binarySearch(arr, target)
  SET lo TO 0
  SET hi TO LENGTH OF arr - 1
  WHILE lo <= hi DO
    SET mid TO (lo + hi) DIV 2
    IF arr[mid] = target THEN
      RETURN mid
    ELSE IF arr[mid] < target THEN
      SET lo TO mid + 1
    ELSE
      SET hi TO mid - 1
    END IF
  END WHILE
  RETURN -1
END FUNCTION

// ── Two-pointer: pair sum in sorted array — O(n) ──────
FUNCTION twoPointerSum(arr, target)
  SET left TO 0
  SET right TO LENGTH OF arr - 1
  WHILE left < right DO
    SET sum TO arr[left] + arr[right]
    IF sum = target THEN
      RETURN [left, right]
    ELSE IF sum < target THEN
      SET left TO left + 1
    ELSE
      SET right TO right - 1
    END IF
  END WHILE
  RETURN NULL
END FUNCTION

// ── Sliding window: max sum subarray of size k — O(n) ─
FUNCTION maxWindowSum(arr, k)
  SET windowSum TO 0
  FOR i FROM 0 TO k - 1 DO
    SET windowSum TO windowSum + arr[i]
  END FOR
  SET maxSum TO windowSum
  FOR i FROM k TO LENGTH OF arr - 1 DO
    SET windowSum TO windowSum + arr[i] - arr[i - k]
    IF windowSum > maxSum THEN
      SET maxSum TO windowSum
    END IF
  END FOR
  RETURN maxSum
END FUNCTION
```

COMMON MISTAKES

- Binary searching an unsorted array — produces wrong results silently; always verify precondition
- Infinite loop in binary search — using `lo = mid` instead of `lo = mid + 1` when mid is not the answer
- Two-pointer on unsorted data — the logic relies on sorted order; use it only when applicable
- Fixed vs variable sliding window — fixed uses index arithmetic; variable expands/shrinks based on a condition

TIPS

- When you see "find minimum/maximum that satisfies condition", think binary search on the answer space
- Two-pointer and sliding window both run in O(n) — recognise them by "pair" or "subarray" in the problem
- Test binary search with edge cases: empty array, single element, target at lo and hi boundaries
- BFS guarantees shortest path in unweighted graphs — if the problem mentions "minimum steps", start with BFS

SUMMARY

- Linear O(n) for unsorted; binary O(log n) requires sorted; hash O(1) avg with preprocessing
- Binary search halves search space each iteration — terminates in $\lceil \log_2 n \rceil$ steps
- Two-pointer — sorted array pair problems in O(n); sliding window — subarray problems in O(n)
- BFS = shortest path (unweighted); DFS = connectivity, cycles, topological order

Divide & Conquer

split · solve subproblems · combine · recurrence relations · Master Theorem · classic examples

THE DIVIDE & CONQUER PARADIGM

Divide & Conquer solves a problem by recursively breaking it into *smaller subproblems* of the same type, solving each independently, and then *combining* the results. Three phases always appear: **Divide** — split the input into smaller pieces; **Conquer** — solve each piece recursively (base case when small enough); **Combine** — merge the sub-answers into the full answer.

The power of D&C comes from reducing work exponentially: a problem of size n splits into subproblems of size n/2, so the recursion tree has O(log n) levels. Analysis uses **recurrence relations**. The **Master Theorem** solves the most common forms in closed form, letting you determine Big O without unrolling the recursion by hand.

MASTER THEOREM

- Form: **T(n) = a·T(n/b) + f(n)** where a ≥ 1, b > 1
- `a` — number of subproblems per level
- `b` — factor by which input shrinks
- `f(n)` — cost of divide + combine step
- **Case 1**: f(n) = O(n^(log_b a - ε)) → T(n) = Θ(n^log_b a) — subproblems dominate
- **Case 2**: f(n) = Θ(n^log_b a) → T(n) = Θ(n^log_b a · log n) — equal work each level
- **Case 3**: f(n) = Ω(n^(log_b a + ε)) → T(n) = Θ(f(n)) — combine step dominates
- Merge sort: T(n)=2T(n/2)+O(n) → Case 2 → O(n log n)

NOTES

- → maxSubarray — divide at midpoint; max subarray is either fully in left half, fully in right half, or crosses mid
- → `maxCrossing` — scan outward from mid both ways; max left-arm + max right-arm = best crossing sum
- → Inversions — during merge, when a right element is placed before a left element, it forms inversions with all remaining left elements
- → Inversion count is zero for sorted arrays and n(n-1)/2 for reverse-sorted arrays
- → Closest pair — brute force is O(n²); D&C finds strip of width 2d around midpoint; only O(1) comparisons per strip point
- → All three examples share T(n) = 2T(n/2) + O(n) → Master Case 2 → O(n log n)

CLASSIC D&C ALGORITHMS

ALGORITHM	RECURRENCE	RESULT
Binary Search	T(n)=T(n/2)+O(1)	O(log n)
Merge Sort	T(n)=2T(n/2)+O(n)	O(n log n)
Quick Sort (avg)	T(n)=2T(n/2)+O(n)	O(n log n)
Max Subarray (D&C)	T(n)=2T(n/2)+O(n)	O(n log n)
Strassen Matrix Mult	T(n)=7T(n/2)+O(n²)	O(n^2.81)
Closest Pair	T(n)=2T(n/2)+O(n)	O(n log n)
Karatsuba Mult	T(n)=3T(n/2)+O(n)	O(n^1.585)
Count Inversions	T(n)=2T(n/2)+O(n)	O(n log n)

D&C VS DYNAMIC PROGRAMMING

DIMENSION	DIVIDE & CONQUER	DYNAMIC PROGRAMMING
Subproblems overlap?	No (independent)	Yes (reused)
Memoization needed?	No	Yes
Direction	Top-down recursion	Top-down or bottom-up
Best fit	Sorting, search, geometry	Optimization, counting
Complexity gain	O(n) → O(n log n) via split	O(2ⁿ) → O(n²) via caching

CLASSIC D&C EXAMPLES IN PSEUDOCODE

```
// ── Maximum subarray — Kadane-by-D&C version ──────────────
// T(n)=2T(n/2)+O(n) → O(n log n)  [Kadane's DP is O(n)]
FUNCTION maxSubarray(arr, lo, hi)
  IF lo = hi THEN RETURN arr[lo] END IF
  SET mid TO (lo + hi) DIV 2
  SET leftMax TO CALL maxSubarray(arr, lo, mid)
  SET rightMax TO CALL maxSubarray(arr, mid+1, hi)
  SET crossMax TO CALL maxCrossing(arr, lo, mid, hi)
  RETURN MAX(leftMax, rightMax, crossMax)
END FUNCTION

FUNCTION maxCrossing(arr, lo, mid, hi)
  SET leftSum TO -∞
  SET sum TO 0
  FOR i FROM mid DOWN TO lo DO
    SET sum TO sum + arr[i]
    IF sum > leftSum THEN SET leftSum TO sum END IF
  END FOR
  SET rightSum TO -∞
  SET sum TO 0
  FOR i FROM mid+1 TO hi DO
    SET sum TO sum + arr[i]
    IF sum > rightSum THEN SET rightSum TO sum END IF
  END FOR
  RETURN leftSum + rightSum
END FUNCTION

// ── Counting inversions (merge-sort variant) ──────────────
// An inversion: i < j but arr[i] > arr[j]
// T(n)=2T(n/2)+O(n) → O(n log n)
FUNCTION countInversions(arr)
  IF LENGTH OF arr <= 1 THEN RETURN [arr, 0] END IF
  SET mid TO LENGTH OF arr DIV 2
  SET [left, lInv] TO CALL countInversions(arr[0..mid-1])
  SET [right, rInv] TO CALL countInversions(arr[mid..end])
  SET [merged, splitInv] TO CALL mergeCount(left, right)
  RETURN [merged, lInv + rInv + splitInv]
END FUNCTION

// ── Closest pair of points — O(n log n) ──────────────────
FUNCTION closestPair(points)
  IF LENGTH OF points <= 3 THEN
    RETURN CALL bruteForce(points)
  END IF
  SET mid TO LENGTH OF points DIV 2
  SET left TO points[0..mid-1]
  SET right TO points[mid..end]
  SET dL TO CALL closestPair(left)
  SET dR TO CALL closestPair(right)
  SET d TO MIN(dL, dR)
  RETURN CALL stripClosest(points, mid, d)
END FUNCTION
```

COMMON MISTAKES

- Missing the combine step — splitting and solving subproblems without merging results gives nothing useful
- Applying D&C when subproblems overlap — use DP with memoization instead (see Sheet 11)
- Wrong base case size — base case must handle the smallest valid input; forgetting size-1 causes infinite recursion
- Misapplying Master Theorem — check a ≥ 1 and b > 1; theorem does not apply to all recurrences

TIPS

- Always ask: "what does the crossing case look like?" — D&C problems often hide work in cross-boundary handling
- Draw the recursion tree — write the work at each level and sum across all log n levels to find the total
- If the combine step is O(n), the total is almost always O(n log n) via Master Case 2
- For geometry problems (closest pair, convex hull), D&C often yields the optimal O(n log n) algorithm

SUMMARY

- D&C: Divide → Conquer (recursive) → Combine; subproblems must be independent
- Master Theorem solves T(n) = a·T(n/b) + f(n) in three cases
- Most D&C algorithms achieve O(n log n) — the efficient middle ground
- When subproblems overlap, D&C becomes DP with memoization (Sheet 11)

Sheet 11

Memoization & Dynamic Programming

overlapping subproblems · optimal substructure · top-down · bottom-up · tabulation · classic patterns

DYNAMIC PROGRAMMING

Dynamic programming (DP) solves problems by breaking them into *overlapping subproblems*, solving each subproblem exactly once, and storing the result so it is never recomputed. It applies when a problem has two properties: **optimal substructure** (optimal solution is built from optimal sub-solutions) and **overlapping subproblems** (same sub-problems recur many times).

There are two equivalent approaches. **Top-down with memoization**: write the natural recursion, cache results in a hash map or array. **Bottom-up tabulation**: fill a DP table from smallest subproblems to largest, avoiding recursion overhead entirely. Both yield the same asymptotic complexity; bottom-up often has better constant factors and avoids stack overflow.

DP CHECKLIST

- **Overlapping subproblems?** — same inputs computed more than once → DP applies
- **Optimal substructure?** — optimal answer built from optimal sub-answers → DP applies
- **Define state** — what changes between subproblems? (index, remaining capacity, etc.)
- **Write recurrence** — express dp[i] or dp[i][j] in terms of smaller states
- **Base cases** — smallest valid state with a known answer
- **Direction** — top-down (recursion + cache) or bottom-up (table iteration)
- **Space optimisation** — many 2D DP tables reduce to O(n) by keeping only the previous row

NOTES

- `IF n IN memo THEN RETURN memo[n]` — cache hit avoids recomputation; converts O(2ⁿ) to O(n)
- Bottom-up fibTab fills dp left to right; each value depends only on the two before it
- Knapsack — 2D state: row = items considered so far, column = remaining capacity
- Knapsack decision per cell: include item i (if it fits) or skip it; take the max of both options
- LCS — when characters match, extend the diagonal; otherwise take the max of left/above
- LCS dp[m][n] = length; backtrack through the table from (m,n) to reconstruct the actual subsequence
- Both knapsack and LCS can reduce to O(n) space by keeping only the previous row at a time

CLASSIC DP PROBLEMS

PROBLEM	STATE	COMPLEXITY
Fibonacci	dp[n]	O(n) time · O(1) space*
Climbing Stairs	dp[i] = dp[i−1]+dp[i−2]	O(n)
0/1 Knapsack	dp[i][w]	O(n·W)
Unbounded Knapsack	dp[w]	O(n·W)
Longest Common Subseq	dp[i][j]	O(m·n)
Longest Increasing Subseq	dp[i]	O(n²) / O(n log n)
Coin Change (min coins)	dp[amount]	O(n·amount)
Edit Distance	dp[i][j]	O(m·n)
Matrix Chain Mult	dp[i][j]	O(n³)

TOP-DOWN VS BOTTOM-UP

DIMENSION	TOP-DOWN (MEMO)	BOTTOM-UP (TAB)
Code style	Recursive — mirrors problem definition	Iterative — explicit loops
Computes	Only needed subproblems	All subproblems in order
Stack risk	Deep recursion → overflow	None
Performance	Cache overhead per call	Better constant factors
Space optimise	Hard (cache must stay full)	Easy — roll arrays
When to use	Natural recursion, sparse states	Dense state space, no overflow

FIBONACCI, 0/1 KNAPSACK & LONGEST COMMON SUBSEQUENCE

```
// ── Fibonacci: naive O(2ⁿ) → memoised O(n) ──────────────
SET memo TO {}
FUNCTION fib(n)
  IF n <= 1 THEN RETURN n END IF
  IF n IN memo THEN RETURN memo[n] END IF
  SET memo[n] TO CALL fib(n-1) + CALL fib(n-2)
  RETURN memo[n]
END FUNCTION

// Bottom-up tabulation — same result, no recursion
FUNCTION fibTab(n)
  IF n <= 1 THEN RETURN n END IF
  SET dp TO array of size n+1
  SET dp[0] TO 0
  SET dp[1] TO 1
  FOR i FROM 2 TO n DO
    SET dp[i] TO dp[i-1] + dp[i-2]
  END FOR
  RETURN dp[n]
END FUNCTION        // O(n) time, O(n) space
                    // Space-opt: keep only prev two vars

// ── 0/1 Knapsack — O(n·W) ──────────────────────────────
// n items each with weight[i] and value[i]; capacity W
// dp[i][w] = max value using first i items, capacity w
FUNCTION knapsack(weights, values, W)
  SET n TO LENGTH OF weights
  SET dp TO 2D array (n+1) × (W+1), all zeros
  FOR i FROM 1 TO n DO
    FOR w FROM 0 TO W DO
      IF weights[i-1] > w THEN
        SET dp[i][w] TO dp[i-1][w]  // can't include item i
      ELSE
        SET dp[i][w] TO MAX(
          dp[i-1][w],               // exclude item i
          dp[i-1][w - weights[i-1]] + values[i-1]  // include
        )
      END IF
    END FOR
  END FOR
  RETURN dp[n][W]
END FUNCTION

// ── Longest Common Subsequence — O(m·n) ────────────────
// dp[i][j] = LCS length of a[0..i-1] and b[0..j-1]
FUNCTION lcs(a, b)
  SET m TO LENGTH OF a
  SET n TO LENGTH OF b
  SET dp TO 2D array (m+1) × (n+1), all zeros
  FOR i FROM 1 TO m DO
    FOR j FROM 1 TO n DO
      IF a[i-1] = b[j-1] THEN
        SET dp[i][j] TO dp[i-1][j-1] + 1
      ELSE
        SET dp[i][j] TO MAX(dp[i-1][j], dp[i][j-1])
      END IF
    END FOR
  END FOR
  RETURN dp[m][n]
END FUNCTION
```

COMMON MISTAKES

- Applying DP when subproblems don't overlap — that's D&C, not DP; memoization adds overhead for no gain
- Incorrect base cases — forgetting dp[0] = 0 in coin change causes wrong answers for all amounts
- Wrong recurrence direction — LCS fills from (1,1) to (m,n); going the wrong way corrupts the table
- Premature space optimisation — reduce to 1D only after the 2D

TIPS

- Start with the recursive brute-force solution, verify it is correct, then add a memo table
- Draw the DP table on paper before coding — filling it by hand reveals the recurrence and fill order
- Most 1D DP problems have a recurrence of the form dp[i] = f(dp[i−1], dp[i−2], …, dp[0])
- Coin change (min coins) and knapsack are template problems —

SUMMARY

- DP applies when: overlapping subproblems + optimal substructure both hold
- Top-down = recursion + cache; bottom-up = fill table iteratively
- Memoization converts naive O(2ⁿ) Fibonacci to O(n) by caching repeated calls
- Most 2D DP tables can be space-optimised to O(n) by keeping only the previous row

Bitwise Operations

AND · OR · XOR · NOT · left/right shift · masks · bit tricks · two's complement

WORKING AT THE BIT LEVEL

Bitwise operations act directly on the binary representation of integers, one bit at a time. They run in O(1) and are among the fastest operations a CPU executes. While rarely needed in high-level application code, bitwise tricks appear constantly in systems programming, embedded systems, competitive programming, compression, cryptography, and low-level optimisation.

All integers in memory are stored in **two's complement** form — the most significant bit is the sign bit (0 = positive, 1 = negative), and negation is achieved by flipping all bits and adding 1. This means bitwise operations on negative numbers can produce surprising results unless you reason carefully about representation.

CORE OPERATORS

- `a AND b` (`&`) — 1 only when both bits are 1; used for masking
- `a OR b` (`|`) — 1 when at least one bit is 1; used for setting bits
- `a XOR b` (`^`) — 1 when bits differ; used for toggling and swap
- `NOT a` (`~`) — flips all bits; ~n = -(n+1) in two's complement
- `a LEFT SHIFT n` (`<<`) — multiply by 2^n; shifts bits left, fills with 0
- `a RIGHT SHIFT n` (`>>`) — divide by 2^n (integer); arithmetic shift preserves sign bit
- **Two's complement** — -x = ~x + 1; enables a single adder circuit for add/subtract
- **Bit mask** — a value used with AND/OR/XOR to isolate, set, or clear specific bits

NOTES

- → `1 LEFT SHIFT i` — creates a mask with only bit i set: 1, 2, 4, 8 … for i = 0,1,2,3
- → `n AND mask` — isolates bits in mask; all other bits become 0 (masking)
- → `n OR mask` — forces bits in mask to 1 without changing other bits (setting)
- → `n AND NOT mask` — forces bits in mask to 0 without changing other bits (clearing)
- → `n AND (n-1)` — clears the lowest set bit; if result is 0, n was a power of 2
- → Kernighan's bit count — each iteration strips one set bit; loops exactly popcount(n) times → O(set bits)
- → XOR swap — works because a XOR a = 0 and 0 XOR b = b; fails if a and b alias the same memory location
- → findUnique — XOR is commutative and associative; pairs cancel to 0; the unique element remains

BITWISE TRUTH TABLE (PER BIT)

A	B	A AND B	A OR B	A XOR B	NOT A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

QUICK-REFERENCE BIT TRICKS

GOAL	EXPRESSION
n is even	`n AND 1 = 0`
n is odd	`n AND 1 = 1`
n × 2	`n LEFT SHIFT 1`
n DIV 2	`n RIGHT SHIFT 1`
n × 2^k	`n LEFT SHIFT k`
Clear all but lowest set bit	`n AND (-n)`
Clear lowest set bit	`n AND (n-1)`
Is power of two?	`n > 0 AND n AND(n-1)=0`
Negate (two's complement)	`NOT n + 1`
Absolute value trick	`mask = n RIGHT SHIFT 31; (n XOR mask) - mask`

COMMON BIT TRICKS IN PSEUDOCODE

```
// ── Check if bit i is set ──────────────────────────────
FUNCTION isBitSet(n, i)
  RETURN (n AND (1 LEFT SHIFT i)) ≠ 0
END FUNCTION
// isBitSet(0b1010, 1) → TRUE  (bit 1 = 1)

// ── Set bit i ──────────────────────────────────────────
FUNCTION setBit(n, i)
  RETURN n OR (1 LEFT SHIFT i)
END FUNCTION

// ── Clear bit i ────────────────────────────────────────
FUNCTION clearBit(n, i)
  RETURN n AND NOT(1 LEFT SHIFT i)
END FUNCTION

// ── Toggle bit i ───────────────────────────────────────
FUNCTION toggleBit(n, i)
  RETURN n XOR (1 LEFT SHIFT i)
END FUNCTION

// ── Check if n is a power of 2 ─────────────────────────
FUNCTION isPowerOfTwo(n)
  RETURN n > 0 AND (n AND (n - 1)) = 0
END FUNCTION
// Powers of 2: ...0001000; n-1 = ...0000111; AND = 0

// ── Count set bits (Brian Kernighan's algorithm) ───────
FUNCTION countBits(n)
  SET count TO 0
  WHILE n ≠ 0 DO
    SET n TO n AND (n - 1)   // clears lowest set bit
    SET count TO count + 1
  END WHILE
  RETURN count
END FUNCTION

// ── XOR swap (no temp variable) ────────────────────────
FUNCTION xorSwap(a, b)
  SET a TO a XOR b
  SET b TO b XOR a
  SET a TO a XOR b
  RETURN [a, b]              // values exchanged
END FUNCTION

// ── Find single non-duplicate in array ─────────────────
// XOR of a number with itself = 0; with 0 = itself
FUNCTION findUnique(arr)
  SET result TO 0
  FOR each n IN arr DO
    SET result TO result XOR n
  END FOR
  RETURN result              // the lone element remains
END FUNCTION

// ── Multiply / divide by powers of 2 ──────────────────
// n LEFT SHIFT k  = n * 2^k
// n RIGHT SHIFT k = n DIV 2^k  (floor division for positive n)
```

COMMON MISTAKES

- Confusing logical operators (AND, OR) with bitwise — logical short-circuits; bitwise processes all bits
- Shifting by more than the integer width — behaviour is undefined in C/C++; avoid shifts ≥ 32 on 32-bit ints
- XOR swap on aliased variables — if a and b point to the same location, both become 0
- Assuming right shift is always unsigned — arithmetic right shift fills with sign bit; logical fills with 0

TIPS

- XOR to find the single non-duplicate element is a classic interview trick — memorise it
- Use bit masks to represent subsets of n items — a number with n bits encodes 2^n possible subsets
- Bitmask DP uses integers as compact sets of boolean flags — allows O(1) set operations
- When you need to check/set a flag, prefer bitwise over a boolean array for cache efficiency

SUMMARY

- AND = mask/isolate; OR = set bits; XOR = toggle/swap/cancel pairs; NOT = flip all
- Left shift ×2^n; right shift ÷2^n — much faster than multiplication/division
- `n AND (n-1)` clears lowest set bit; `n AND (-n)` isolates it
- All bitwise ops are O(1) — use them to replace O(n) loops when working with flags or subsets

Design Principles

DRY · KISS · YAGNI · SOLID · separation of concerns · coupling · cohesion · code smells

WHY DESIGN PRINCIPLES MATTER

Design principles are heuristics distilled from decades of software engineering experience. They do not guarantee correct programs, but they guide toward code that is *easier to understand*, *cheaper to change*, and *less likely to break* when requirements evolve.

The three foundational rules — **DRY** (Don't Repeat Yourself), **KISS** (Keep It Simple, Stupid), and **YAGNI** (You Aren't Gonna Need It) — apply at every scale, from a single function to a distributed system. The **SOLID** principles extend these ideas specifically to object-oriented design, describing how classes and modules should be structured and how they should depend on each other. Together they define what "clean code" looks like in practice.

SOLID PRINCIPLES

LETTER	PRINCIPLE	IN PLAIN ENGLISH
S	Single Responsibility	A class should have one reason to change — do one thing well
O	Open / Closed	Open for extension (add new behaviour), closed for modification (don't break existing code)
L	Liskov Substitution	A subclass must be substitutable for its parent without breaking the program
I	Interface Segregation	Clients should not depend on interfaces they don't use — many small interfaces beat one large one
D	Dependency Inversion	High-level modules depend on abstractions, not concrete implementations; inject dependencies

PRINCIPLES APPLIED

SITUATION	PRINCIPLE	ACTION
`Same calculation in 3 places`	DRY	Extract to a named function
`100-line function`	KISS / SRP	Break into smaller named functions
`Building a feature "just in case"`	YAGNI	Delete it; add when needed
`Subclass breaks parent's contract`	LSP	Rethink the hierarchy; use composition
`Adding a case to every switch`	OCP	Use polymorphism / strategy pattern
`Hard-coded dependency on a DB`	DIP	Inject a repository interface
`Unused interface methods`	ISP	Split the interface

DRY · KISS · YAGNI

- **DRY** — every piece of knowledge should have one authoritative source; duplication makes changes risky
- **DRY violation** — copy-pasted code, two functions that do the same thing, magic numbers repeated across files
- **KISS** — prefer the simplest solution that works; complexity is a liability, not an asset
- **KISS violation** — premature abstraction, over-engineered class hierarchies, clever one-liners that take 10 minutes to parse
- **YAGNI** — don't build features for imagined future requirements; build what is needed now
- **YAGNI violation** — "we might need a plugin system someday", configuration options no one uses, dead code paths
- All three tensions balance: DRY can conflict with KISS when abstraction adds complexity; YAGNI can conflict with DRY when deduplication requires future-proofing

COUPLING & COHESION

CONCEPT	GOOD	BAD
`Coupling`	Low — modules interact through narrow, stable interfaces	High — modules depend on each other's internal details
`Cohesion`	High — everything in a module relates to one purpose	Low — a class does unrelated things ("God class")
`Goal`	Low coupling + high cohesion = modules that change for one reason and can be tested in isolation	

COMMON CODE SMELLS

SMELL	INDICATES
`Long Method`	Too many responsibilities — extract sub-functions
`God Class`	Low cohesion — split into focused classes
`Feature Envy`	Method uses another class's data more than its own — move it
`Magic Numbers`	Unnamed constants — extract to named constants
`Shotgun Surgery`	One change requires edits in many files — consolidate
`Data Clumps`	Same 3+ parameters always appear together — create a struct/class

SEPARATION OF CONCERNS

LAYER / CONCERN	RESPONSIBILITY
`Presentation`	Render output; handle user input (UI, HTML, CLI)
`Business Logic`	Domain rules, calculations, decisions (pure functions)
`Data Access`	Read/write to DB, files, APIs (repository/adapter layer)
`Infrastructure`	Logging, caching, messaging, config

COMPOSITION OVER INHERITANCE

PREFER	BECAUSE
`Composition (has-a)`	Flexible — behaviour assembled at runtime; easy to test and swap
`Inheritance (is-a)`	Use only for true is-a relationships with stable hierarchies

COMMON MISTAKES

- Over-applying DRY — not all similar code is duplicate; premature deduplication creates wrong abstractions
- Misusing SOLID as rules — they are heuristics; violating one deliberately for clarity is sometimes correct
- Gold-plating (YAGNI violation) — adding configurable plugin architectures when a simple function suffices
- Deep inheritance (LSP/OCP risk) — prefer composition; deep trees make substitution and testing brittle

TIPS

- The best code is the code you don't write — YAGNI keeps codebases small and maintainable
- Name things by what they *do* or *are*, not by how they work — names reveal intent
- If a function needs a comment to explain what it does, the name is wrong — rename it
- Write code for the reader, not the computer — the computer doesn't care; your future self does

SUMMARY

- DRY — one source of truth; KISS — simplest solution; YAGNI — build what's needed now
- SOLID — five principles for OO design: SRP, OCP, LSP, ISP, DIP
- High cohesion + low coupling = modules that change for one reason, testable in isolation
- Code smells signal design problems — they don't prove a bug but do invite future ones

Problem Solving Patterns

pattern recognition · brute force → optimise · two pointers · sliding window · fast-slow · greedy · backtracking

A UNIVERSAL PROBLEM-SOLVING PROCESS

Every algorithmic problem — from a coding interview to a production bottleneck — responds to the same structured approach. Jumping straight to code without understanding the problem is the single most common source of wrong answers and wasted time.

1. Understand — restate the problem in your own words, identify inputs, outputs, constraints, and edge cases. **2. Explore** — work through small examples by hand; let the pattern emerge. **3. Brute force** — find any working solution first; correctness before efficiency. **4. Optimise** — identify the bottleneck; apply the appropriate pattern. **5. Code** — implement cleanly. **6. Test** — run examples including edge cases; trace through the code.

NOTES

- Fast-slow (Floyd's) — slow moves 1 step, fast moves 2; if a cycle exists they meet inside it
- Fast-slow also finds middle of list (slow at mid when fast reaches end), and start of cycle
- Greedy — always pick the interval that ends earliest; locally optimal choices produce a globally optimal result here
- Greedy correctness requires an exchange argument proof — greedy doesn't always work; verify before using it
- Backtracking — choose an option, recurse, undo the choice; explores the full decision tree
- `removeLast` (backtrack step) — restores state so the next loop iteration starts fresh
- Monotonic stack — pop elements smaller than current; when popped, current is their "next greater"; O(n) because each element pushed and popped at most once

PATTERN RECOGNITION TRIGGERS

- **"Find pair / subarray"** → two pointers or sliding window
- **"Shortest path / min steps"** → BFS (unweighted) or Dijkstra (weighted)
- **"All combinations / permutations"** → backtracking
- **"Optimum value (max/min)"** → DP or greedy
- **"Top K / K closest"** → heap (priority queue)
- **"Sorted array, find target"** → binary search
- **"Count / frequency"** → hash map
- **"Cycle detection in list/graph"** → fast-slow pointers or visited set
- **"Tree / graph traversal"** → DFS (recursive) or BFS (queue)
- **"Overlapping subproblems"** → dynamic programming with memoization

PATTERN CATALOGUE

PATTERN	TYPICAL COMPLEXITY	KEY REQUIREMENT
Brute Force	O(n²)–O(n!)	Correctness baseline; always start here
Two Pointers	O(n)	Sorted array or list; pair constraint
Sliding Window	O(n)	Contiguous subarray/substring
Fast & Slow	O(n)	Linked list / cycle detection
Binary Search	O(log n)	Sorted data or monotonic predicate
BFS / DFS	O(V+E)	Graph or tree traversal
Heap / Priority Queue	O(n log k)	Top-k, streaming min/max
Hash Map	O(n)	Frequency, existence, complement lookup
Greedy	O(n log n)*	Local optimal = global optimal (provable)
Backtracking	O(2ⁿ)–O(n!)	All valid combinations / permutations
Dynamic Programming	O(n²) typical	Overlapping subproblems + opt. substructure
Divide & Conquer	O(n log n)	Independent subproblems + combine step

BRUTE FORCE → OPTIMISED

BRUTE FORCE	BOTTLENECK	OPTIMISED
Nested loop over array	O(n²)	Two pointers or hash map → O(n)
Recompute window sum	O(n·k)	Sliding window → O(n)
Linear scan of sorted array	O(n)	Binary search → O(log n)
Recursive fib / DP	O(2ⁿ)	Memoization → O(n)
Check all pairs for sum	O(n²)	Hash complement → O(n)
Sort then scan	O(n²) naive sort	Merge/quick sort → O(n log n)
Find k-th largest by sort	O(n log n)	Heap of size k → O(n log k)

KEY PATTERNS IN PSEUDOCODE

```
// ── Fast-slow pointers: detect cycle in linked list ──────────
FUNCTION hasCycle(head)
  SET slow TO head
  SET fast TO head
  WHILE fast ≠ NULL AND fast.next ≠ NULL DO
    SET slow TO slow.next
    SET fast TO fast.next.next
    IF slow = fast THEN RETURN TRUE END IF
  END WHILE
  RETURN FALSE
END FUNCTION              // Floyd's algorithm — O(n) O(1) space
// ── Greedy: activity selection (interval scheduling) ─────────
// Maximise number of non-overlapping intervals
FUNCTION maxActivities(intervals)
  CALL sort(intervals, by end time ascending)
  SET count TO 1
  SET lastEnd TO intervals[0].end
  FOR i FROM 1 TO LENGTH OF intervals - 1 DO
    IF intervals[i].start >= lastEnd THEN
      SET count TO count + 1
      SET lastEnd TO intervals[i].end
    END IF
  END FOR
  RETURN count
END FUNCTION              // O(n log n) sort + O(n) scan
// ── Backtracking: generate all subsets ──────────────────────
FUNCTION subsets(nums)
  SET result TO []
  CALL backtrack(nums, 0, [], result)
  RETURN result
END FUNCTION

FUNCTION backtrack(nums, start, current, result)
  CALL result.append(COPY OF current)    // record current subset
  FOR i FROM start TO LENGTH OF nums - 1 DO
    CALL current.append(nums[i])         // include nums[i]
    CALL backtrack(nums, i + 1, current, result)
    CALL current.removeLast()            // undo (backtrack)
  END FOR
END FUNCTION
// ── Monotonic stack: next greater element ────────────────────
FUNCTION nextGreater(arr)
  SET result TO array of -1, same length as arr
  SET stack TO []                        // stores indices
  FOR i FROM 0 TO LENGTH OF arr - 1 DO
    WHILE NOT stack.isEmpty() AND arr[stack.top()] < arr[i] DO
      SET idx TO stack.pop()
      SET result[idx] TO arr[i]
    END WHILE
    stack.push(i)
  END FOR
  RETURN result
END FUNCTION              // O(n) — each index pushed/popped once
```

COMMON MISTAKES

- Optimising before verifying correctness — a fast wrong answer is worse than a slow right one
- Skipping edge cases — empty input, single element, all duplicates, and max/min values should always be tested
- Applying greedy without proof — many problems look greedy but require DP; verify with an exchange argument
- Choosing backtracking when DP applies — backtracking explores O(2ⁿ) states; DP prunes to polynomial with memoization

TIPS

- Always start with brute force — it clarifies the problem, gives you a test oracle, and often reveals the pattern to optimise
- Ask "what information is being recomputed?" — the answer almost always points to the right optimisation (cache, precompute, or reduce search space)
- When stuck, try a smaller example — a 3-element array is easier to trace than a 100-element one
- Recognise the pattern first, then recall the algorithm — pattern → algorithm → implementation is faster than algorithm → brute-force discovery

SUMMARY

- Process: Understand → Explore → Brute Force → Optimise → Code → Test
- Pattern triggers in the problem statement reveal the right algorithm before you write any code
- Most O(n²) brute forces become O(n) with two pointers, hash map, or sliding window
- Fast-slow pointers, greedy, and backtracking each solve specific structural problem types